

WiredTiger Read-Ticket Exhaustion on MongoDB

From Linux I/O fundamentals to the root cause and fix

FTDC analysis — server ~~qa6-mongo-global-11-b~~ (z3-highmem-44, Titanium local SSD)

Incident: 2026-06-06 (UTC)

Contents

Executive summary	1
Part I — The concepts	3
1. The Linux page cache and buffered I/O	3
2. Dirty pages and writeback	3
3. The dirty-page thresholds (and what they are a percentage <i>of</i>)	3
4. The writeback flusher: how many workers?	4
5. fsync and forced flushing	4
6. The block layer: one queue, reads versus writes	4
7. WiredTiger essentials	5
Part II — The incident	6
8. Environment and symptom	6
9. Evidence and investigation	6
10. Root cause	8
11. Why the tickets hit <i>exactly</i> zero	9
12. The fix	10
13. “Will it still flush in a burst?”	10
14. Rollout and verification	11
Appendix A — Methodology	12
Appendix B — Full validation table	12
Appendix C — Glossary of key metrics	13

Executive summary

On 6 June the MongoDB primary repeatedly ran out of WiredTiger **read tickets**: the available read-ticket count fell to 0 in recurring bursts and the queue of waiting reads grew to **2,510** at its worst (04:54:21 UTC). Application read latency spiked during each burst.

The investigation showed that the SSD itself was healthy throughout — device read latency held at ~0.11 ms (exactly what Titanium local SSD should deliver). The drops were caused by **reads queueing behind write bursts at the block-device layer**. Those write bursts are the **Linux kernel flushing accumulated dirty page-cache to disk** roughly every 30 seconds. As a read-heavy workload grew

~8x overnight, WiredTiger fell behind on eviction and checkpointing, its dirty cache and history store ballooned (history store grew from 0.1 GB to 12 GB), and that produced ever-larger dirty-page piles that the kernel dumped to disk in violent ~1-second bursts. Each burst momentarily saturated the device write queue; cache-miss reads issued during that window stalled, held their tickets, and the 128-ticket pool emptied.

Two diagnoses were tested and one earlier hypothesis was corrected:

- It is **not** the SSD (read latency flat at 0.11 ms).
- It is **not** a WiredTiger checkpoint `fsync` (the checkpoint-fsync counters stay flat during every burst). This corrected an earlier assumption.
- It **is** the kernel's periodic dirty-page writeback. The kernel dirty-page counter (`/proc/meminfo Dirty`) sawtooths — climbing ~2.8 GB over ~30 s, then collapsing in ~1 s — and that collapse correlates 0.976 with the device write burst.

A per-event cross-check confirmed the mechanism for **every** drop: all 28 read-ticket-zero intervals coincide with a writeback burst (Appendix B).

The fix is therefore Linux VM and block-layer tuning to smooth writeback, plus reducing the WiredTiger write volume at the source. A faster disk would not help — the device is already the fastest available and its reads are not the bottleneck.

This document first explains the underlying OS and storage-engine concepts (Part I), then walks through the incident and applies those concepts to the evidence, root cause, and fix (Part II).

Part I — The concepts

1. The Linux page cache and buffered I/O

When a process reads or writes a normal file, it does not talk to the disk directly. The kernel keeps a **page cache** in RAM: file data is cached in memory in 4 KB pages. A read that finds its page in the cache is served from RAM in microseconds (a “cache hit”); a miss triggers a physical read from the device.

Writes work the same way but asymmetrically. A normal buffered `write()` copies data into a page-cache page, marks that page **dirty**, and returns immediately — the data is in RAM, not yet on disk. The kernel later writes dirty pages to the device in the background. This is called **write-back caching**, and it is why writes appear fast: durability is deferred.

WiredTiger uses buffered I/O for its data files (it does not use `O_DIRECT` by default). Every page WiredTiger writes during eviction or checkpoint therefore lands first in the Linux page cache as a dirty page, and the kernel — not WiredTiger — decides when those pages physically reach the SSD. This single fact is central to the incident.

2. Dirty pages and writeback

A “dirty” page is a page-cache page whose contents have been modified in RAM but not yet persisted to disk. The kernel must eventually write dirty pages back to the device; this process is called **writeback**.

Writeback is triggered in three ways:

1. **Periodic (time-based)**. A flusher thread wakes on a fixed interval and writes back pages that have been dirty longer than an expiry age. This bounds how long data sits only in RAM.
2. **Background (volume-based)**. When the total amount of dirty memory crosses a *background* threshold, the kernel wakes the flushers to write asynchronously, while applications keep running unthrottled.
3. **Foreground / throttled (hard limit)**. When dirty memory crosses a higher *hard* threshold, processes that try to dirty more pages are themselves throttled (made to sleep / help with write-back) until dirty drops. This is back-pressure on the writer.

The key intuition: if dirty pages are allowed to accumulate into a large pile and are then released all at once, the device sees a sudden flood of writes — a burst. If instead writeback runs early and continuously, the same total volume reaches the device as a smooth trickle.

3. The dirty-page thresholds (and what they are a percentage of)

Six `sysctl` knobs govern writeback. The values observed on this server:

<code>sysctl</code>	value on this host	meaning
<code>vm.dirty_background_ratio</code>	10	start background writeback when dirty exceeds 10% of dirtyable memory
<code>vm.dirty_background_bytes</code>	0 (disabled)	absolute-bytes form of the above; 0 means the ratio is used
<code>vm.dirty_ratio</code>	20	throttle writers when dirty exceeds 20% of dirtyable memory

sysctl	value on this host	meaning
<code>vm.dirty_bytes</code>	0 (disabled)	absolute-bytes form of the hard limit
<code>vm.dirty_expire_centisecs</code>	3000 (30 s)	a dirty page becomes eligible for periodic writeback after 30 s
<code>vm.dirty_writeback_centisecs</code>	500 (5 s)	the flusher wakes every 5 s to do periodic writeback

The critical and widely misunderstood detail: the ratios apply to **dirtyable memory**, not total RAM. Dirtyable memory is approximately free memory plus the reclaimable file-backed page cache. Anonymous memory — which includes WiredTiger’s ~221 GB cache (a heap allocation) — is **excluded**.

On this 363 GB host, dirtyable memory was only ~110 GB, so the effective thresholds were ~11 GB (background) and ~22 GB (hard), not 36 GB / 73 GB as a naive “percent of RAM” would suggest. This matters for the diagnosis, as we will see.

4. The writeback flusher: how many workers?

Writeback is performed per backing device (per-bdi), not per CPU. Since the old global `pdflush` pool was removed (~kernel 2.6.32), each block device has its own flusher context served by a dedicated writeback workqueue — effectively **one flusher working a given device at a time**.

The consequence: writeback for a single large file is serialized through one writeback context. You do not get 44 parallel flushers fanning a flush across cores; you get one stream per device. That is part of why a large flush arrives as a concentrated burst rather than a wide, smooth spread.

5. fsync and forced flushing

`fsync(fd)` (and `fdatasync`, `msync`) forces all dirty pages of a file to the device and waits until the device confirms durability. Databases call `fsync` at well-defined commit points to guarantee durability. WiredTiger calls `fsync` on its data files at the end of each checkpoint, and the journal is synced frequently for durability.

An `fsync` is a **synchronous flush barrier**: it can submit a large batch of writes at once and blocks the caller until they complete. So both periodic writeback and `fsync` can produce write bursts; the difference is the trigger (a timer/threshold versus an explicit durability call).

6. The block layer: one queue, reads versus writes

Below the page cache, I/O requests enter a per-device queue and are dispatched to the device. On NVMe the default I/O scheduler is usually **none** — a single shared queue with no prioritization between reads and writes. If thousands of write requests are queued and a read arrives, that read waits behind the writes. The device’s raw service time may be microseconds, but the **queueing delay** can be tens of milliseconds.

Two mechanisms protect reads from write floods:

- **I/O schedulers** such as `mq-deadline` (separate read/write queues, read expiry, a `writes_starved` cap) or `kyber` (targets a read-latency goal). These let reads jump ahead of queued writes.

- **Writeback throttling (WBT)**, `wbt_lat_usec`. The block layer monitors read latency and throttles *background* buffered writeback when reads start to suffer. It is purpose-built for exactly the “writes starving reads” problem.

The takeaway for Part II: a large write burst on a shared, unprioritized queue directly degrades concurrent reads, even on a fast SSD.

7. WiredTiger essentials

A few WiredTiger mechanisms tie the OS concepts to MongoDB:

Cache and eviction. WiredTiger keeps a large in-memory cache (here, 221 GB). When the cache fills, **eviction** writes modified (“dirty”) pages out and drops clean pages to make room. Eviction is done by dedicated worker threads; if they cannot keep up, application threads are recruited or throttled.

Dirty cache. The portion of the WiredTiger cache holding modified pages. WiredTiger targets a low dirty fraction (default `eviction_dirty_target` 5%, `eviction_dirty_trigger` 20%). A rising dirty fraction means eviction is falling behind.

History store. To serve snapshot/MVCC reads and support rollback, WiredTiger keeps prior versions of documents in an on-disk **history store**. Under heavy update churn — or when checkpoints cannot advance fast enough — the history store grows, and it must itself be written and checkpointed.

Checkpoints. Every ~60 seconds WiredTiger writes a consistent on-disk checkpoint: it flushes dirty pages (and history-store content) to the data files (buffered writes, into the page cache) and then `fsyncs`. A bigger dirty set and history store mean a heavier checkpoint.

Concurrency tickets. WiredTiger admits only a bounded number of operations into the storage engine at once: by default **128 read tickets and 128 write tickets**. An operation must hold a ticket for the entire time it is executing inside WiredTiger — including any time it spends blocked waiting to read a page in from disk. When all 128 read tickets are held, new reads wait in a queue. Therefore: anything that makes in-flight reads block — such as a page-in stuck behind a device write burst — holds tickets and can exhaust the pool.

Part II — The incident

8. Environment and symptom

The affected node is a replica-set **primary** on a GCP **z3-highmem-44** instance: 44 vCPUs, 363 GB RAM, and Titanium **local SSD** (the fastest disk tier available, microsecond-class read latency). The WiredTiger cache is configured at 221 GB. The data volume is a 6-device NVMe stripe (`nvme1n1–nvme6n1`).

The symptom: WiredTiger `concurrentTransactions.read.available` (read tickets free, out of 128) repeatedly dropped to 0 across 6 June, in bursts that grew more frequent and severe from ~02:35 and peaked between 04:05 and 05:05. The read queue reached 2,510 waiting operations. Over the same window the read workload grew about 8x.

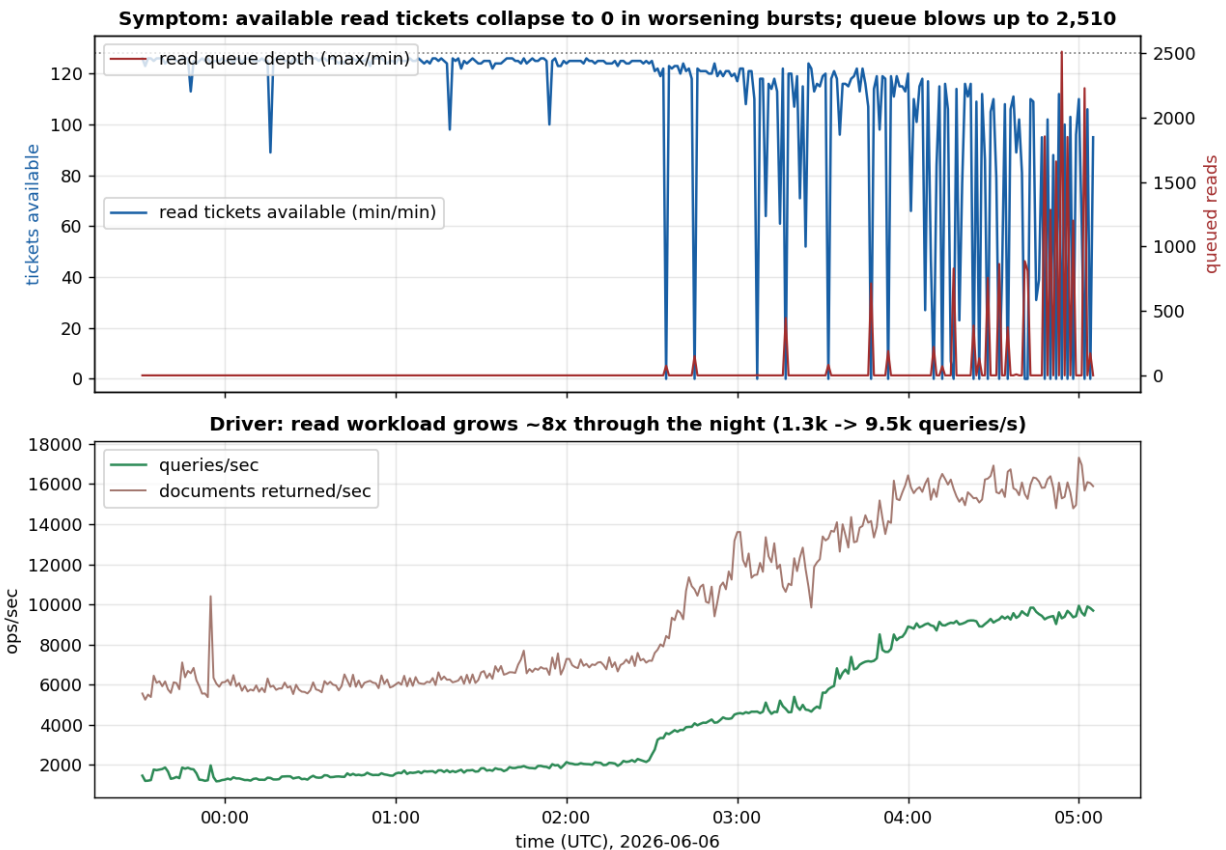


Figure 1: Symptom and driver: read tickets collapse to zero in worsening bursts while the read queue blows up (top); the read workload grows roughly eightfold across the window (bottom).

9. Evidence and investigation

9.1 The SSD is not the bottleneck

Per-I/O device **read** latency held at ~0.11 ms median (0.13 ms p90) for the entire window — exactly what local SSD should deliver. There was no period of slow reads. An apparent “15 ms read” seen early in the analysis was an arithmetic artifact: during a stalled second, total queued time was divided by the handful of reads that managed to *complete*, which inflates the per-read figure. The device’s actual read service time never degraded.

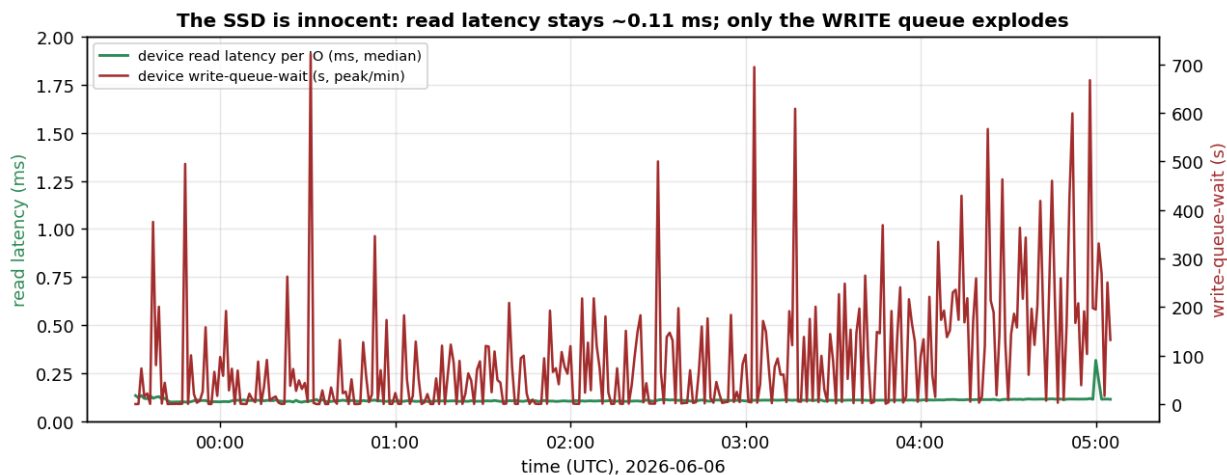


Figure 2: The SSD is innocent: device read latency stays around 0.11 ms throughout, while only the write-queue wait explodes during the incident.

9.2 Reads were starving behind write bursts

On the seconds when read tickets hit 0, the device **write-queue** wait time jumped from a baseline of ~ 81 ms to a median of **122,485 ms** (summed across the stripe), while read latency stayed flat. In other words, reads were not slow — they were waiting in line behind a flood of writes.

At the worst burst (04:54), the device write count spiked to **94,504 writes/second** for one second, with write-queue wait exceeding 160,000 ms and one disk in the stripe pegged at 89% busy. **Reads that completed that second had effectively waited ~ 15 ms in the queue, and** the read-ticket pool emptied.

9.3 It was *not* a WiredTiger checkpoint fsync (correction)

A natural first hypothesis was that the burst was a checkpoint fsync. The FTDC data **refutes** this: the WiredTiger counters `transaction fsync calls for checkpoint`, `transaction fsync duration for checkpoint`, and `connection.total fsync I/Os` all stay flat during every storm — `connection.total fsync I/Os` sits at ~ 10 /s straight through the 94,504-writes/second burst. Whatever dumps the writes to the device, it is not a WiredTiger checkpoint fsync. This corrected the earlier assumption.

9.4 It *was* the kernel’s dirty-page writeback

FTDC captures `systemMetrics.memory.Dirty_kb` directly from `/proc/meminfo`. Kernel dirty pages form a clean **sawtooth**: they build to ~ 2.8 GB over ~ 30 seconds, then collapse to ~ 0.1 GB in 1–2 seconds. Each collapse coincides exactly with a device write burst — the correlation between device-write MB/s and the per-second drop in kernel dirty pages is **0.976**. The read-ticket drop at 04:54:21 sits precisely inside one of these collapses.

The ~ 30 -second period matches `vm.dirty_expire_centisecs` (3000 cs = 30 s). Because WiredTiger concentrates its writes in a few large data files, each file’s accumulated dirty pages cross the 30 s expiry age together and are flushed in one pulse.

9.5 Why the kernel let ~ 3 GB accumulate

Recall the thresholds act on *dirtyable* memory, not total RAM. From `/proc/meminfo` on this host:

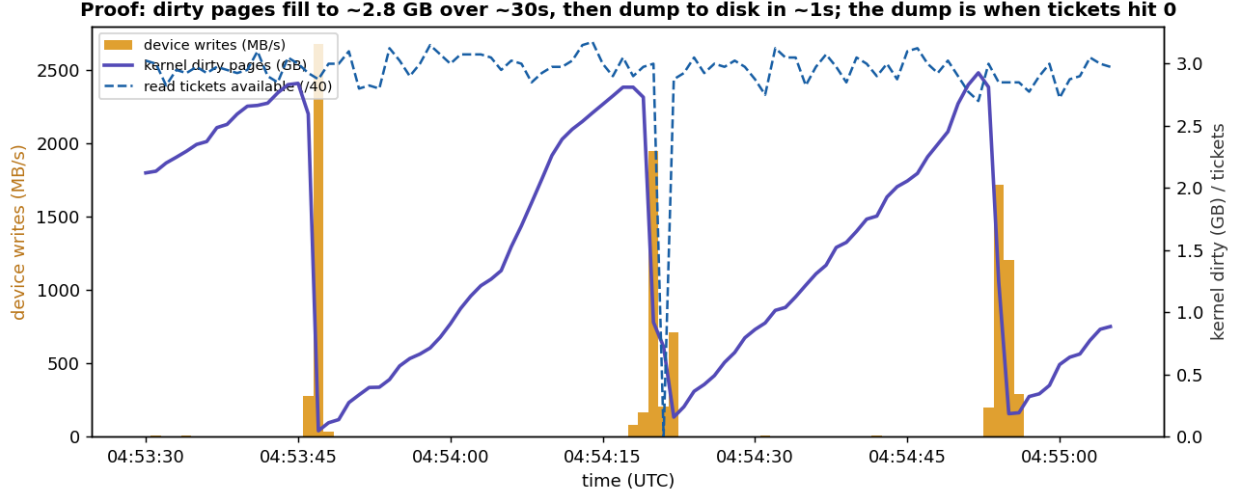


Figure 3: Proof: kernel dirty pages fill to about 2.8 GB over ~30 s, then dump to the SSD in ~1 s. The largest dump (04:54:20–21) is the exact second read tickets fall to zero (dashed line).

memory metric	value
MemTotal	362.7 GB
MemAvailable	116.3 GB
Cached (page cache)	107.8 GB
Active(file) + Inactive(file)	~108 GB
~dirtyable memory (free + file LRU)	~110 GB
background threshold (10% of dirtyable)	~11 GB
hard throttle (20% of dirtyable)	~22 GB
observed peak kernel dirty during incident	4.2 GB

Dirty peaked at only 4.2 GB — well below the ~11 GB background threshold — so the **volume** trigger never fired. The flushes were therefore the time-based 30 s expiry, releasing the whole pile at once. And because dirty never approached the ~22 GB hard limit, WiredTiger’s threads were never write-throttled; the read stalls were purely block-layer queueing during the flush.

9.6 Validation: every drop aligns with a flush burst

Before recommending any change, every read-ticket-zero event was cross-checked against the writeback signal. There were 29 zero-seconds, forming 28 distinct intervals. **All 28 (100%)** coincide with a device write burst within a ± 2 s window; in every case kernel dirty pages are falling at that moment. The full per-event table is in Appendix B. This rules out an unrelated second cause and confirms the mechanism applies to the entire incident.

10. Root cause

Putting the chain together: a sustained ~8x growth in read traffic pushed WiredTiger past the rate at which eviction and checkpointing could keep up. The dirty cache rose from 0.2% to ~6% and the history store grew from 0.1 GB to 12 GB. WiredTiger streamed all of that out as buffered writes into the Linux page cache, where dirty pages accumulated to 2–3 GB and were then released by the kernel’s 30 s periodic writeback in violent ~1-second bursts. Each burst saturated the shared block-device write

queue; cache-miss reads issued during the burst stalled behind the writes, held their read tickets for the duration, and the 128-ticket pool emptied — producing `available = 0` and a queue of up to 2,510 waiting reads.

It is a feedback loop: more load to more dirty/history-store to bigger flush bursts to deeper read stalls.

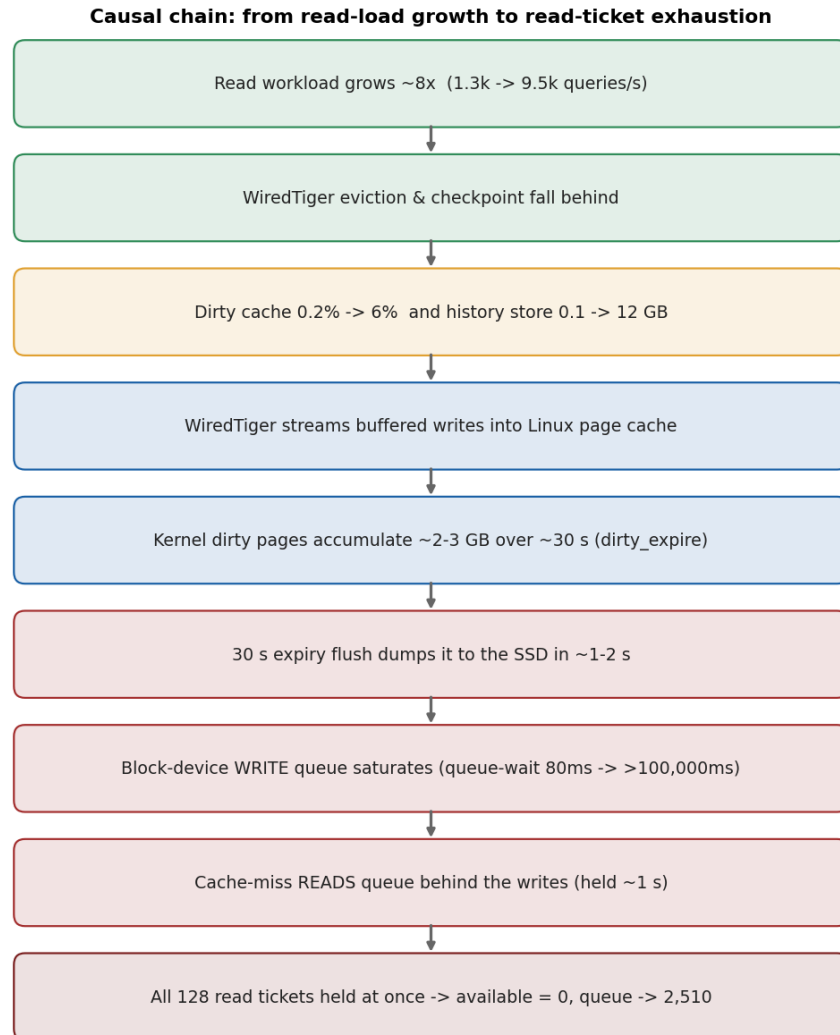


Figure 4: The causal chain from the read-load surge down to read-ticket exhaustion.

11. Why the tickets hit *exactly* zero

A useful sanity check via concurrency math (Little’s law): under steady state, average tickets-in-use equals arrival rate times average hold time. At baseline, ~4 tickets were in use serving ~1,250 reads/s — a ~3 ms effective hold. During a flush burst, in-flight reads block on a page-in stuck behind the write queue for up to ~1 second. With reads arriving at ~9,500/s, even a few hundred milliseconds of synchronized stall on all in-flight reads is enough to pin all 128 tickets simultaneously and pile thousands more into the queue. The burst does not need to last long; it only needs to stall the reads that currently hold tickets, all at once. That is exactly what a 1-second device write saturation does.

Note one direction of the relationship: every ticket drop has a flush burst, but not every flush burst caused a drop (there were ~440 burst-seconds and only 29 zero-seconds). The drops occurred when

a burst landed on top of already-high read concurrency, later in the window. Smoothing the bursts removes the trigger; reducing the read load removes the precondition.

12. The fix

The disk is already the fastest tier, and its reads are not the problem, so faster hardware is not the answer. Three layers of change address the trigger, protect reads, and reduce the source.

12.1 Kernel VM tuning (primary — runtime, no restart)

Make writeback start early and run continuously so dirty never accumulates into a multi-GB pile:

```
vm.dirty_background_bytes    = 268435456    # 256 MB    (was ~11 GB via the 10% ratio)
vm.dirty_bytes               = 2147483648    # 2 GB      (was ~22 GB via the 20% ratio)
vm.dirty_expire_centisecs    = 500          # 5 s       (was 30 s)
vm.dirty_writeback_centisecs = 100          # 1 s       (was 5 s)
```

Setting the `*_bytes` forms automatically zeroes the corresponding `*_ratio`. Absolute bytes are strongly preferred on large-RAM hosts because the ratio drifts with page-cache size and is unpredictable. The effect: kernel dirty hovers near ~256 MB with shallow, near-continuous writeback instead of a 2.8 GB sawtooth, so the device write queue stays shallow and reads stop queueing behind a wall of writes. (Lowest-risk first step: change only `dirty_background_bytes` and leave the hard limit at default.)

12.2 Block layer (protect reads even if a burst forms — runtime, per device)

```
# for each data device nvme1n1..nvme6n1:
echo mq-deadline > /sys/block/nvme1n1/queue/scheduler
# confirm writeback throttling is active (protects read latency):
cat /sys/block/nvme1n1/queue/wbt_lat_usec      # non-zero; tune ~2000-5000 us
```

`none` (the NVMe default) gives writes equal footing with reads on one shared queue; `mq-deadline` prioritizes reads and ages writes separately. `kyber` with a read-latency target is a good alternative. WBT throttles background writeback when it detects read latency rising.

12.3 WiredTiger / MongoDB (durable root-cause fix — rolling restart)

Keep the dirty set and history store smaller so there is less to flush:

```
storage:
  wiredTiger:
    engineConfig:
      configString: "eviction_dirty_target=2,eviction_dirty_trigger=5,
                    eviction=(threads_min=4,threads_max=8)"
```

Equally important, investigate the upstream drivers: why the history store grew to 12 GB (check secondary replication lag and the majority-commit / stable-timestamp progress, and pinned timestamps via `db.serverStatus().wiredTiger.transaction`), and what drove the ~8x read-traffic growth (and whether it can be capped, smoothed, or routed to secondaries).

13. “Will it still flush in a burst?”

A fair objection: if WiredTiger dumps, say, 1 GB of writes, won’t the kernel still flush 1 GB at once even with a low `dirty_background_bytes`? The flusher does **not** rate-limit a drain on its own — if a

1 GB pile exists, it pushes it to the device as fast as the device accepts. The fix works by preventing the pile from forming, via three effects:

1. WiredTiger does not write 1 GB atomically; it streams many small buffered `pwrite()` calls, so dirty *ramps* rather than steps.
2. With a low background threshold the flusher starts at ~256 MB and runs continuously, draining at WiredTiger's production rate. On this host WiredTiger's physical write rate averaged ~24 MB/s (468 GB over 5.5 h) against an SSD that drains at multiple GB/s, so dirty pins near the threshold.
3. Between the background and hard thresholds, the kernel paces the **producer**: `balance_dirty_pages()` inserts small sleeps into WiredTiger's writes so the dirty rate matches the device's writeback bandwidth. Dirty stabilizes instead of running away.

So with these settings dirty will not accumulate to 1 GB and then dump; it stays near 256 MB and flushes continuously. WBT and the I/O scheduler are the backstop if any residual burst still forms.

14. Rollout and verification

Apply the VM and scheduler changes on one **secondary** first (they are non-disruptive and instantly revertible), observe for a few hours, then roll across the set with the **primary last**. Persist the sysctls in `/etc/sysctl.d/` and the scheduler choice in a udev rule (the `/sys/block` setting resets on reboot). Make the WiredTiger `configString` change during the same rolling restart.

Verify success by watching, during peak load:

- `grep -e Dirty -e Writeback /proc/meminfo` in a loop — dirty should hover near a few hundred MB, not sawtooth to GBs.
- `iostat -x 1` — write `aqu-sz` spikes should shrink and read `r_await` should stay flat.
- FTDC — `concurrentTransactions.read.available` should stop hitting 0 and the read queue should stay near 0.

Appendix A — Methodology

The analysis used the server’s FTDC (Full-Time Diagnostic Data Capture) archive in `diagnostic.data`. FTDC files are concatenated BSON documents; metric chunks are zlib-compressed and delta-encoded. A self-contained decoder reconstructed every metric at 1-second resolution across the 6 June files (a 20,019-sample window from 23:31 the prior night through 05:05). Counters were converted to per-second rates by differencing; gauges were used directly. Device metrics come from `systemMetrics.disks.*` (sourced from `/proc/diskstats`) and `systemMetrics.memory.*` (from `/proc/meminfo`); storage-engine metrics come from `serverStatus.wiredTiger.*`.

Key cross-checks: read-queue depth correlates most strongly with application-thread disk page-in time ($r = 0.79$) and read latency ($r = 0.77$); device write MB/s correlates with the kernel dirty-page flush at $r = 0.976$; and 28 of 28 read-ticket-zero intervals align with a writeback burst.

Appendix B — Full validation table

Every read-ticket-zero interval and its concurrent writeback evidence (baseline: device writes ~ 0.8 MB/s, write-queue-wait ~ 81 ms). “Dirty flushed” is the drop in kernel dirty pages in the ± 2 s window.

#	time (UTC)	read queue peak	write-queue- wait peak (ms)	dirty flushed (GB)	aligned
1	02:35:26	75	120,430	0.22	Yes
2	02:45:03	150	160,102	0.28	Yes
3	03:07:20	2	149,360	0.36	Yes
4	03:17:11	445	608,612	0.23	Yes
5	03:32:07	82	226,668	0.30	Yes
6	03:47:48	713	368,857	1.20	Yes
7	03:53:48	189	240,831	0.86	Yes
8	04:09:44	221	138,246	1.12	Yes
9	04:12:51	71	235,800	1.45	Yes
10	04:16:34	829	163,553	1.02	Yes
11	04:23:03	386	210,666	0.88	Yes
12	04:25:42	132	51,568	0.15	Yes (zoom-confirmed)
13	04:28:20-21	757	257,309	1.44	Yes
14	04:28:47	723	463,163	1.48	Yes
15	04:32:20	864	184,521	1.13	Yes
16	04:35:01	373	157,985	1.59	Yes
17	04:41:13	886	177,606	1.40	Yes
18	04:42:50	811	131,531	1.92	Yes
19	04:48:02	618	191,372	2.03	Yes
20	04:48:36	1,853	259,467	1.38	Yes
21	04:50:47	1,284	161,846	1.56	Yes
22	04:52:57	1,659	599,169	0.79	Yes
23	04:54:21	2,510	207,807	1.81	Yes
24	04:56:30	1,849	191,224	1.85	Yes
25	04:58:49	1,200	667,492	0.80	Yes
26	05:02:15	2,227	200,722	0.65	Yes

#	time (UTC)	read queue peak	write-queue- wait peak (ms)	dirty flushed (GB)	aligned
27	05:02:43	1,780	268,855	1.51	Yes
28	05:04:49	171	250,788	1.91	Yes

Result: 28/28 intervals (100%) align with a device write burst; the single low-magnitude case (#12) was confirmed by inspection (write-queue-wait jumped from ~150 ms to ~47,000 ms while dirty fell 0.80 -> 0.65 GB across two seconds).

Appendix C — Glossary of key metrics

- `wiredTiger.concurrentTransactions.read.available / .out / .queueLength`: free read tickets (of 128), tickets in use, and operations queued waiting for one.
- `wiredTiger.cache.tracked dirty bytes in the cache`: modified data held in the WiredTiger cache awaiting eviction/checkpoint.
- `wiredTiger.cache.bytes` belonging to the history store table in the cache: in-cache size of the MVCC history store.
- `wiredTiger.transaction.transaction fsync calls/duration for checkpoint`: WiredTiger's own checkpoint fsync activity (flat during the bursts — proof the bursts were not checkpoint fsyncs).
- `systemMetrics.memory.Dirty_kb`: kernel dirty page-cache size from `/proc/meminfo` — the sawtooth that proves the writeback mechanism.
- `systemMetrics.disks.<dev>.io_queued_ms / write_time_ms / read_time_ms / reads / writes`: per-device `/proc/diskstats` fields used to separate read latency (flat) from write-queue saturation (spiking).